Week 5 Worksheet (Solutions)

COMP2048

Semester 1, 2026

For teaching staff use only. Please do not distribute to students yet.

Task 1. *Derivations and Language of a Grammar*

Consider the following CFG G over $\Sigma = \{a, b\}$:

$$S \to aSb \mid aSbb \mid \varepsilon$$

(a) Derive the strings `ab`, `abb`, and `aabbb` using $\Rightarrow$ notation. Show each substitution step.

(b) Draw the parse tree for your derivation of `aabbb`.

(c) Is the string `abbb` in $L(G)$? Is the string `aabb` in $L(G)$? Justify each answer by giving a derivation or explaining why no derivation exists.

(d) Describe $L(G)$ in set-builder notation, e.g. $L(G) = \{\, a^n b^n \mid n \geq 0 \,\}$.

Solution.

(a) Three derivations:

`ab`:

$$S \Rightarrow aSb \Rightarrow a\varepsilon b = ab$$

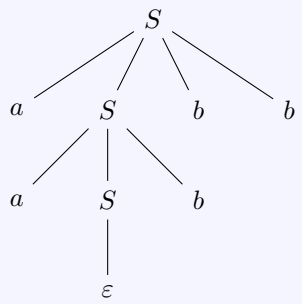
`abb`:

$$S \Rightarrow aSbb \Rightarrow a\varepsilon bb = abb$$

`aabbb`: We use $S \to aSbb$ first, then $S \to aSb$, then $S \to \varepsilon$:

$$S \Rightarrow aSbb \Rightarrow aaSbbb \Rightarrow aa\varepsilon bbb = aabbb$$

(b) Parse tree for `aabbb`:



The root $S \to aSbb$ gives children a, S, b, b. The inner $S \to aSb$ gives children a, S, b. The innermost $S \to \varepsilon$ gives child ε. Reading the leaves left to right: a, a, ε, b, b, b $= aabbb$.

(c) `abbb`: **No**, $abbb \notin L(G)$.

Each rule application adds exactly one a and either one b (via $S \to aSb$) or two b's (via $S \to aSbb$). After n rule applications before the final $S \to \varepsilon$, the string has n a's and between n and $2n$ b's. The string $abbb$ has 1 a and 3 b's, but with $n = 1$ we can produce at most $2(1) = 2$ b's. Since $3 > 2$, no derivation exists.

`aabb`: **Yes**, $aabb \in L(G)$. Derivation:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\varepsilon bb = aabb$$

(d) $L(G) = \{\, a^n b^m \mid n \le m \le 2n,\ n \ge 0 \,\}$.

Justification. Suppose a derivation uses the rule $S \to aSb$ exactly k_1 times and $S \to aSbb$ exactly k_2 times (then terminates with $S \to \varepsilon$). The resulting string has:

- $n = k_1 + k_2$ copies of a,
- $m = k_1 + 2k_2$ copies of b.

Since $m = n + k_2$ and $0 \le k_2 \le n$, we get $n \le m \le 2n$.

Conversely, for any $n \le m \le 2n$, set $k_2 = m - n$ and $k_1 = 2n - m$. Both are non-negative, and $k_1 + k_2 = n$, so this is achievable.

Task 2. *Ambiguity*

Consider the following CFG over $\Sigma = \{a, b\}$:

$$S \to aSb \mid bSa \mid SS \mid \varepsilon$$

(a) Derive the string `abab`.

(b) Find two *distinct* parse trees for `abab`. Explain why this confirms the grammar is ambiguous.

(c) What language does this grammar generate? Describe it in plain English.

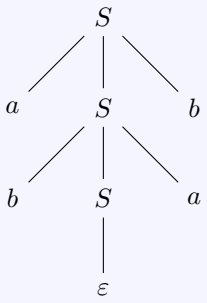
Solution.

(a) One derivation of `abab`:

$$S \Rightarrow aSb \Rightarrow a(bSa)b \Rightarrow ab\varepsilon ab = abab$$

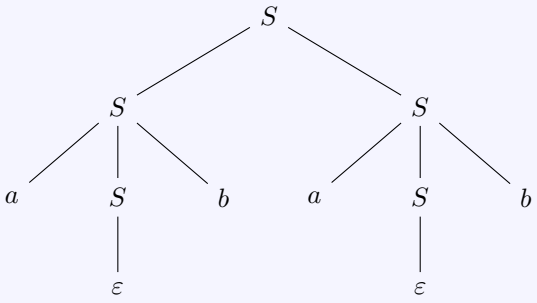
(b) Two distinct parse trees for `abab`:

Tree 1 (root rule $S \to aSb$):



Leaves: a, b, ε, a, b = `abab`. Uses $S \to aSb$ at the root, then $S \to bSa$, then $S \to \varepsilon$.

Tree 2 (root rule $S \to SS$):



Leaves: a, ε, b, a, ε, b = `abab`. Uses $S \to SS$ at the root, then $S \to aSb$ for each child, then $S \to \varepsilon$ for each.

These two parse trees are distinct (the root rule is $S \to aSb$ in Tree 1 and $S \to SS$ in Tree 2). Since the string `abab` has two different parse trees, the grammar is ambiguous by definition.

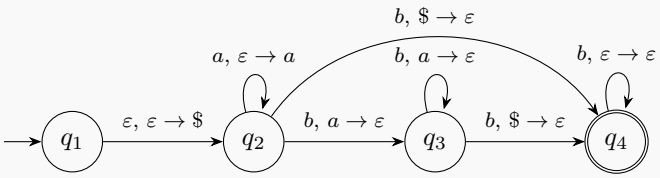
(c) The grammar generates all strings over $\{a, b\}$ with **equal numbers of a's and b's**.

Why every generated string is balanced: Each rule preserves the a/b balance. $S \to aSb$ adds one a and one b. $S \to bSa$ adds one b and one a. $S \to SS$ concatenates two balanced strings. $S \to \varepsilon$ produces the empty string (trivially balanced).

Why every balanced string is generated: By induction on string length. The empty string is generated by $S \to \varepsilon$. For a non-empty balanced string w: if w starts with a, it must contain a matching b somewhere; one can show w can be decomposed as aSb or SS for appropriate balanced substrings, each of which is generated by the induction hypothesis.

Task 3. *PDA Tracing*

Consider the following PDA P with input alphabet $\Sigma = \{a, b\}$, stack alphabet $\Gamma = \{a, \$\}$, start state q_1, and accepting state q_4.



(a) Trace the PDA on input `abb`. Record the state and stack contents after each step in a table like the one started below. Does the PDA accept or reject?

Step	Read	State	Action	Stack (top first)
0	–	$q_1 \to q_2$	$\varepsilon, \varepsilon \to \$$: push \$	\$
1				
⋮				

(b) Trace the PDA on input `aabb`. Does the PDA accept or reject?

(c) Trace the PDA on input `bbb`. Does the PDA accept or reject?

(d) Describe the language recognised by this PDA in set-builder notation (as in Task 1d) or plain English.

Solution.

(a) Trace on `abb`:

Step	Read	State	Action	Stack (top first)
0	–	$q_1 \to q_2$	$\varepsilon, \varepsilon \to \$$: push \$	\$
1	a	q_2	$a, \varepsilon \to a$: push a	a, \$
2	b	$q_2 \to q_3$	$b, a \to \varepsilon$: pop a	\$
3	b	$q_3 \to q_4$	$b, \$ \to \varepsilon$: pop \$	(empty)

End of input in q_4 (accepting). **Accept.**

(b) Trace on `aabb`:

Step	Read	State	Action	Stack (top first)
0	–	$q_1 \to q_2$	$\varepsilon, \varepsilon \to \$$: push \$	\$
1	a	q_2	$a, \varepsilon \to a$: push a	a, \$
2	a	q_2	$a, \varepsilon \to a$: push a	a, a, \$
3	b	$q_2 \to q_3$	$b, a \to \varepsilon$: pop a	a, \$
4	b	q_3	$b, a \to \varepsilon$: pop a	\$

End of input in q_3 with \$ still on the stack. The transition $q_3 \to q_4$ requires reading a b, but there is no more input. q_3 is not an accepting state. **Reject.**

(c) Trace on `bbb`:

Step	Read	State	Action	Stack (top first)
0	–	$q_1 \to q_2$	$\varepsilon, \varepsilon \to \$$: push \$	\$
1	b	$q_2 \to q_4$	$b, \$ \to \varepsilon$: pop \$	(empty)
2	b	q_4	$b, \varepsilon \to \varepsilon$: stay	(empty)
3	b	q_4	$b, \varepsilon \to \varepsilon$: stay	(empty)

End of input in q_4 (accepting). **Accept.**

Note: at step 1, the stack top is \$ (not a), so the $q_2 \to q_3$ transition (b, $a \to \varepsilon$) does not apply. Instead, the $q_2 \to q_4$ transition (b, $\$ \to \varepsilon$) fires.

(d)

$$L(P) = \{\, a^i b^j \mid 0 \le i < j \,\}.$$

In words: all strings of zero or more a's followed by b's, where the number of b's is **strictly greater** than the number of a's.

Why: The PDA pushes one a per input a. Each b in the matching phase ($q_2 \to q_3$, q_3 loop) pops one a. To reach q_4, the PDA must read one additional b that pops the \$ marker (either from q_3 or directly from q_2 if there were no a's). Any further b's are consumed by the q_4 self-loop. So the PDA accepts exactly when the number of b's exceeds the number of a's by at least one.

Task 4. *Designing a CFG*

(a) Design a CFG that generates the language $L = \{a^n b^{2n} \mid n \ge 0\}$.

(b) Write a CFG that generates the same language as the regular expression $0^*1\,(0 \cup 1)^*$.

Hint: Start by describing in plain English what strings this regular expression matches.

(c) Is every regular language also context-free? Briefly explain why or why not.

Solution.

(a)

$$S \to aSbb \mid \varepsilon$$

Each application of $S \to aSbb$ adds one a and two b's. After n applications followed by $S \to \varepsilon$, the string is $a^n b^{2n}$.

Verification:

- $n = 0$: $S \Rightarrow \varepsilon$. ✓
- $n = 1$: $S \Rightarrow aSbb \Rightarrow a\varepsilon bb = abb$. ✓
- $n = 2$: $S \Rightarrow aSbb \Rightarrow aaSbbbb \Rightarrow aa\varepsilon bbbb = aabbbb$. ✓

(b) The regular expression $0^*1(0 \cup 1)^*$ matches all binary strings that contain **at least one** `1`. (The 0^* matches any leading 0's, 1 matches the first 1, and $(0 \cup 1)^*$ matches everything after.)

A CFG:

$$S \to 0S \mid 1A \qquad\qquad A \to 0A \mid 1A \mid \varepsilon$$

S generates zero or more leading 0's (via $S \to 0S$), then produces a 1 and hands off to A (via $S \to 1A$). A generates any string over $\{0, 1\}$ (via $A \to 0A \mid 1A \mid \varepsilon$).

Verification:

- `1`: $S \Rightarrow 1A \Rightarrow 1\varepsilon = 1$. ✓
- `01`: $S \Rightarrow 0S \Rightarrow 01A \Rightarrow 01$. ✓
- `110`: $S \Rightarrow 1A \Rightarrow 11A \Rightarrow 110A \Rightarrow 110$. ✓
- `0`: $S \Rightarrow 0S$, but S always eventually needs $S \to 1A$, so `0` cannot be generated. ✓

(c) **Yes.** Every regular language is also context-free.

A DFA is simply a PDA that never uses its stack. Since every regular language is recognised by some DFA, and every DFA is a (trivial) PDA, every regular language is recognised by some PDA. Since PDAs recognise exactly the context-free languages, every regular language is context-free.

The containment is strict: $\{0^n 1^n \mid n \geq 0\}$ is context-free (we saw a CFG and PDA for it in lecture) but not regular (by the pumping lemma).